

Reviewer Pack — Matroid Rank Inverse Proportional to ACC^0 Circuit Size for ...

Entry 74050dd8b5ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 04:25:11 UTC

Matroid Rank Inverse Proportional to ACC^0 Circuit Size for GF(2) Polynomials

Entry ID: 74050dd8b5ed **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 04:25:11 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): ACC^0 Circuit Size
Statement:

For any GF(2)-polynomial f over n variables, the rank r of its matroid representation satisfies $\text{size}_{\text{ACC}^0}(f) \geq \Omega(n^{1 + 1/r})$

Rationale (proposer’s reasoning):

Matroid rank captures combinatorial dependencies in GF(2) polynomials, while ACC^0 circuit size reflects algebraic complexity. Their inverse proportionality could reveal structural constraints on circuit depth for dependency-rich functions.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9f11ea0f97609ee4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def generate_gf2_polynomial(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def matroid_rank(poly):
    n = int(math.log2(len(poly)))
    rank = 0
    basis = []
    for i in range(n):
        if poly[i] == 1:
            basis.append(i)
            rank += 1
            for j in range(i + 1, n):
                if poly[j] == 1 and any((poly[j] & (1 << k)) != 0 for k in basis):
                    poly[j] ^= (1 << i)
    return rank

def acc0_circuit_size(poly):
    n = int(math.log2(len(poly)))
    size = 0
    for i in range(n):
        if poly[i] == 1:
            size += 1
            for j in range(i + 1, n):
                if poly[j] == 1 and any((poly[j] & (1 << k)) != 0 for k in range(i)):
                    size += 1
    return size

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    c = 1.0 # Constant to be adjusted
    instances_tested = 30
    conjecture_holds = True
    counterexample = ""

    for _ in range(instances_tested):
        poly = generate_gf2_polynomial(n)
        rank = matroid_rank(poly)
        size = acc0_circuit_size(poly)
        if size < c * n**(1 + 1/rank):
            conjecture_holds = False
            counterexample = f"Rank {rank}, Size {size}"
```

```

        break

    return {
        "metric_name": "acc0_circuit_size",
        "metric_value": acc0_circuit_size(poly),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2**i + 1 for i in range(5, 8)] # First 3 prime powers greater than 40

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results; insufficient data to evaluate support fraction or counterexamples | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108380
2	propose	ollama_remote	qwen3:8b	0	0	110158
3	preregistration	ollama_remote	qwen3:8b	0	0	26015
4	novelty	ollama_remote	qwen3:8b	0	0	20626
5	novelty	ollama_remote	qwen3:8b	0	0	13758
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10220
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8409
8	judge	ollama_remote	qwen3:8b	0	0	68750

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 366318 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/74050dd8b5ed.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/74050dd8b5ed.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/74050dd8b5ed.tar.gz` (if generated)